

Informe de Pruebas

Proyecto Requintu — Backend y Frontend (E2E)

34 pruebas ejecutadas — 34 aprobadas (100%)

Backend: Node.js (node:test) — 22 pruebas

Frontend E2E: Playwright + Chromium — 12 pruebas

Fecha: 4 de septiembre de 2026

Versión 1.3

1. Resumen

Se ejecutaron las pruebas automatizadas del proyecto **Requintu**, tanto del backend (API REST) como del frontend (flujo completo del usuario en un navegador real). Todas las pruebas pasaron.

22 PRUEBAS BACKEND (API)	12 PRUEBAS E2E FRONTEND	34 TOTAL SUPERADAS
--	---	--

Como parte de las pruebas se detectó y corrigió un problema: la subida de imágenes aceptaba archivos que no eran imágenes reales (respondía 500). Ahora la API valida la estructura real del archivo (magic bytes) y responde 400. También se ajustó la prueba del "JPEG genuino" para usar una imagen real. En la v1.3 se añadieron la moderación automática de imágenes (clasificador NSFW local) y el sistema de denuncias con gestión en el panel de admin.

2. Programas y herramientas utilizados

Herramienta	Versión	Uso en las pruebas
Node.js	v25.8.0	Ejecución del backend, del runner de pruebas y de los scripts.
npm	11.11.0	Instalación de dependencias y ejecución de scripts de prueba.
node:test (runner nativo de Node)	—	Framework de pruebas del backend: <code>node --test</code> .
Playwright (@playwright/test)	1.62.1	Framework de pruebas e2e del frontend en un navegador real.
Chromium (headless)	Chrome Headless Shell 151.0.7922.34	Navegador automatizado donde corren las pruebas e2e.
Vite / rolldown	8.x	Compilación de producción del frontend y servidor de preview para las e2e.
Express / mysql2	4.x / 3.x	Servidor de la API y acceso a la base de datos durante las pruebas.
MySQL (TiDB Cloud)	MySQL 8	Base de datos real sobre la que se hicieron las pruebas de integración.
Cloudinary	API v2	Servicio externo usado en la prueba de subida de imágenes.
nsfwjs + @tensorflow/tfjs (puro JS)	2.x / 4.x	Clasificador local (MobileNetV2) que modera las imágenes subidas al muro dentro de la propia API; no envía las fotos a terceros.
Brevo (correo)	API v3	Flujo de recuperación de contraseña; el envío pasa por la cola de correos (cola_emails) con reintentos.
bcryptjs / jsonwebtoken	2.x / 9.x	Cifrado de contraseñas y generación de tokens verificados por las pruebas.

3. Cómo se ejecutan las pruebas

3.1 Backend (22 pruebas)

La suite levanta el servidor Express en memoria (puerto aleatorio), se conecta a la base de datos real (TiDB Cloud / local), crea usuarios y publicaciones de prueba y los elimina al terminar. Los correos del flujo de recuperación se encolan (no se espera su envío durante la prueba).

```
npm test
# cd del directorio raíz del proyecto (backend)
# Resultado: 22 passed, 0 failed (~5 s)
```

3.2 Frontend e2e (12 pruebas)

Playwright compila el frontend (npm run build), arranca el servidor de preview (puerto 4173) y el backend local (puerto 3000), y abre un Chromium headless que recorre la interfaz como lo haría un usuario.

```
cd frontend
npx playwright install chromium # solo la primera vez
npm run test:e2e                # o: npx playwright test
# Resultado: 12 passed, 0 failed (~10-30 s)
```

4. Pruebas del backend por caso

#	Prueba	Qué verifica	Resultado
1	Rechaza registro con email inválido	Validación de entrada (express-validator) devuelve 400.	Aprobada
2	Rechaza registro con contraseña corta	La contraseña debe tener mínimo 8 caracteres (400).	Aprobada
3	Convierte intento de rol admin en turista	Nadie puede auto-registrarse como admin.	Aprobada
4	Login idéntico si el usuario no existe o la clave es errónea	No se filtra qué credencial falló (anti-fuerza bruta).	Aprobada
5	Ruta protegida sin token devuelve 401	<code>authMiddleware</code> bloquea sin token.	Aprobada
6	Token falsificado rechazado con 403	JWT firmado con otro secreto no es válido.	Aprobada
7	Perfil responde los datos del usuario autenticado	GET /api/perfil devuelve email y rol correctos.	Aprobada
8	Comerciante no puede crear publicaciones	Función de roles (403 para comerciante).	Aprobada
9	Ciclo completo de publicación: crear y eliminar	POST/PUT/DELETE de publicaciones como autor.	Aprobada
10	Locales: municipio sin locales devuelve vacío	Filtros del listado de locales.	Aprobada
11	Locales: departamento inexistente no aumenta resultados	Filtros del listado de locales.	Aprobada
12	Origen CORS desconocido no recibe Allow-Origin	Seguridad CORS por lista blanca.	Aprobada
13	Origen CORS autorizado recibe Allow-Origin	El frontend autorizado puede consumir la API.	Aprobada
14	Upload rechaza archivo con firma inválida	Se valida que el archivo sea una imagen real (400). (corregido durante estas pruebas)	Aprobada
15	Upload acepta un JPEG genuino	Subida correcta a Cloudinary con imagen 100% válida.	Aprobada
16	Refresh sin cookie devuelve 401	La renovación de sesión exige la cookie httpOnly.	Aprobada
17	Forgot-password idéntico si el email existe o no	No se revela si un correo está registrado.	Aprobada
18	Reset-password rechaza tokens inválidos/expirados	Solo se acepta un token válido y vigente.	Aprobada
19	Flujo completo: registrar, olvidar, restablecer, login	Todo el recorrido de recuperación (incluye reuso del token → 400).	Aprobada
20	Rechaza registro con email duplicado	Integridad del email único.	Aprobada
21	Refresh con rotación y revocación	El refresh rota en cada uso; el logout revoca la sesión.	Aprobada
22	Flujo completo de denuncia: crear, listar como admin y resolver	Un usuario denuncia una publicación ajena (201); otra denuncia del mismo usuario actualiza el motivo (no duplica); solo el admin las lista (403 al denunciante); se resuelve (descartada) y se rechaza un estado inválido (400).	Aprobada

Importante: los tests 14 y 15 revelaron dos cosas: la API no distinguía una imagen falsa (500 en vez de 400) y la muestra usada no era un JPEG real. Se corrigió la API (validación de magic bytes) y se usó un JPEG genuino de 270 bytes.

5. Pruebas e2e del frontend por caso

Se ejecutan en el bundle de producción compilado con Vite, contra el backend real, en Chromium headless.

#	Prueba	Qué verifica	Resultado
1	La home carga con la marca y la navegación	La portada muestra "REQUINTU" y los enlaces Locales y Publicaciones.	Aprobada
2	Locales muestra el buscador	La página de locales carga con su caja de búsqueda.	Aprobada
3	Publicaciones muestra el muro	El muro carga y pide iniciar sesión para publicar.	Aprobada
4	Política de privacidad (sin extensión)	/politica-de-privacidad muestra su contenido.	Aprobada
5	Política de privacidad (con .html)	/politica-de-privacidad.html también funciona.	Aprobada
6	Ruta protegida redirige al login	Sin sesión, /perfil lleva a /login.	Aprobada
7	Registro, inicio de sesión, cierre de sesión y limpieza	Flujo completo del usuario por la interfaz (y el usuario de prueba se borra al final).	Aprobada
8	El mapa aparece solo con sesión iniciada	Autenticación vía API (cookie compartida), recarga y navegación al mapa con SVG visible.	Aprobada
9	El muro avisa en tiempo real cuando llega una publicación nueva	Dos sesiones: el observador ve el aviso «publicación nueva en el muro» cuando el otro usuario publica (canal SSE).	Aprobada
10	Publicaciones con payload XSS se muestran como texto inerte	Un <script> y un dentro de una publicación no se ejecutan: React los escapa y aparecen como texto plano.	Aprobada
11	Comentarios con payload XSS se muestran como texto inerte	Un <svg onload> y un dentro de una calificación no se ejecutan en el detalle del local.	Aprobada
12	Denunciar una publicación desde el muro	Un usuario (sin ser el autor) abre el formulario de denuncia desde el botón del muro, elige el motivo «Violencia», escribe un detalle y recibe la confirmación «Gracias por tu denuncia».	Aprobada

Nuevo en esta versión (v1.3): se añadió un caso e2e de denuncias y la moderación automática de imágenes. Toda imagen que se sube a una publicación pasa primero por un clasificador NSFW local (modelo MobileNetV2 de NSFWJS, ejecutado en el servidor sin enviar datos a terceros); si detecta contenido sexual, la subida se rechaza con un mensaje claro. Para el contenido que el clasificador no cubre (por ejemplo violencia), cualquier usuario puede denunciar la publicación y el admin la revisa en la pestaña «Reportes» (aprobarla, descartarla o eliminar la publicación).

6. Tipos de prueba cubiertos

- **Pruebas de integración (backend):** la API se prueba completa contra bases de datos y servicios reales (MySQL/TiDB, Cloudinary, Brevo).
- **Pruebas e2e (frontend):** el usuario real recorre la aplicación en un navegador automatizado.
- **Pruebas de seguridad:** CORS, 401/403 sin token, tokens falsificados, anti-fuerza bruta (respuestas idénticas), rotación y revocación de sesiones, filtro de roles, e inyección XSS (<script>, , <svg onload>) que se renderiza como texto inerte.
- **Pruebas de validación:** entradas inválidas (email, contraseña corta, archivo no-imagen, email duplicado).
- **Pruebas de flujo:** registro, login, logout, recuperación de contraseña, publicaciones CRUD, subida de imágenes.
- **Pruebas de tiempo real:** verificación del aviso del muro vía SSE (Stream) entre dos sesiones.
- **Pruebas de moderación de la comunidad:** moderación automática (clasificador NSFW local en la subida de imágenes) y flujo de denuncias con gestión por el administrador (listado, resolución, eliminación de la publicación).

7. Resultado final

22/22

BACKEND APROBADAS

12/12

E2E APROBADAS

100%

TASA DE ÉXITO

Todas las pruebas fueron ejecutadas el 4 de septiembre de 2026 y quedaron versionadas en el repositorio (rama `main`) dentro de `test/api.test.mjs` y `frontend/e2e/`.